RUNGTA INTERNATIONAL SKILLS UNIVERSITY

SCHOOL OF INFORMATION TECHNOLOGY

Bachelor of Computer Application (BCA)

PROJECT REPORT

PS No. 9 - Simple Banking System

Language: Python 3

Submitted by	Tej Prakash Nishad
ERP No.	RU 26 01071
Course	BCA
College	Rungta International Skills University

1. Introduction

This project is a console-based Simple Banking System written in Python 3. It lets customers open an account, deposit money, withdraw money, transfer money to another account and check their balance. All account data is stored in a file (**bank.txt**) so that it remains available after the program is closed.

2. Problem Statement

Create a banking application that lets customers open an account, deposit, withdraw, transfer money and check their balance, with all data stored in a file. Account numbers start from 1001. A minimum balance of Rs.500 must always remain. Concepts used: Loops, Functions, Conditions and File Handling.

3. Objectives

• Understand the problem and define it clearly.

• Develop the solution with proper logic using functions, loops, conditions and file handling.

• Create accounts with unique account numbers starting from 1001.

• Enforce the minimum balance rule of Rs.500 and validate all inputs.

• Test the project with normal, boundary, invalid and alternate valid inputs.

4. Requirements

Item	Details
Language	Python 3 (tested on Python 3.14)
Libraries	Standard library only: json, math, pathlib
Editor	Visual Studio Code
Data file	bank.txt (JSON format)
Execution	python one.py (from the console / VS Code terminal)

5. Features

Menu	Feature	Description
1	Create Account	Takes name and opening deposit (minimum Rs.500); gives a unique account number from 1001
2	Deposit	Adds a positive amount to an existing account
3	Withdraw	Withdraws money; Rs.500 must always remain
4	Transfer	Moves money between two different existing accounts
5	Balance Enquiry	Shows account number, name and balance
6	Exit	Closes the program

6. Design

The program is divided into small functions. Each function does one job, which makes the code easy to read, test and debug.

Function	Purpose
load_accounts()	Reads all accounts from bank.txt (returns empty data if file is missing or corrupt)
save_accounts()	Writes all accounts to bank.txt
amount_input()	Repeats until a valid positive number is entered
account_input() / name_input()	Validate account number and name
create_account()	Opens a new account with unique number
deposit_money() / withdraw_money()	Deposit and withdraw with minimum balance check
transfer_money()	Transfer between two accounts with all checks
balance_enquiry()	Displays account details
main()	Menu loop that calls the above functions

Algorithm (main flow)

• Start the program and load accounts from bank.txt.

• Show the menu and read the option.

• Option 1-5: run the matching function, validate inputs, update and save bank.txt.

• Invalid option: show an error and show the menu again.

• Option 6: print a thank-you message and stop.

Validation rules

• Amount must be a number greater than 0 (letters, blank, zero, negative values are rejected).

• Account number must be digits only and must exist in the file.

• Name cannot be empty; opening deposit must be at least Rs.500.

• Withdrawal or transfer is rejected if balance would fall below Rs.500.

• Sender and receiver in a transfer must be different accounts.

7. Test Cases and Results

All four test cases were run and each produced the expected output.

Test Case	Input	Expected Output	Result
Test Case 1: Normal Input	Menu option 1, Name: Anil, Opening deposit: 2000	Account created. A/C No: 1001 | Balance: 2000	PASS
Test Case 2: Boundary / Minimum Input	Account 1001 has balance 2000; Withdraw 1500	Withdrawal successful. Balance: 500	PASS
Test Case 3: Invalid Input	Account 1001 has balance 2000; Transfer 100 to account 9999	Account 9999 not found. Transfer cancelled.	PASS
Test Case 4: Different Valid Scenario	Accounts 1001 (2000), 1002 (3000); Transfer 1000 from 1002 to 1001; Balance enquiry: 1001	1002 balance: 2000 | 1001 balance: 3000	PASS

Automatic test run (python test_banking.py)

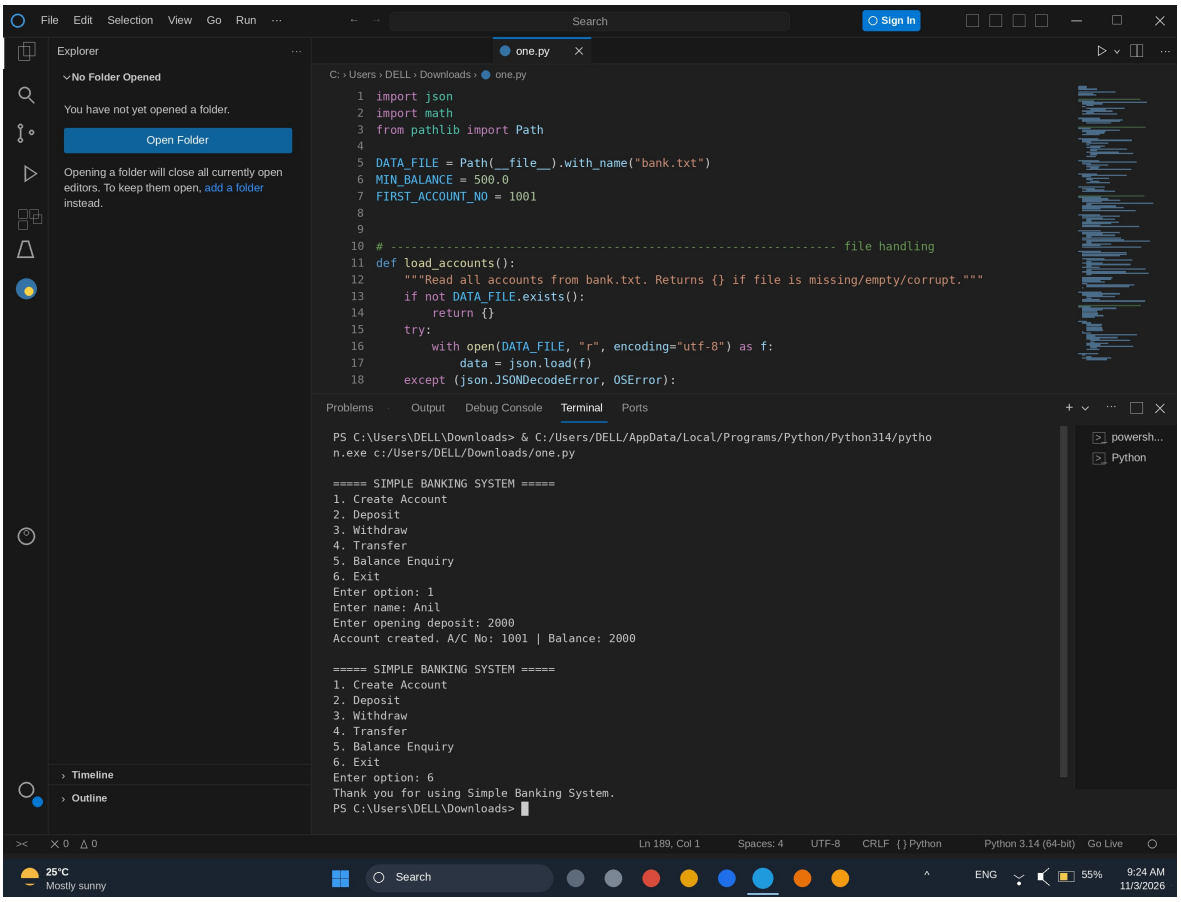
```
test_case_1_normal_input (test_case_1_normal_input)
Create account: Name Anil, opening deposit 2000. ... ok
test_case_2_boundary_minimum_input (test_case_2_boundary_minimum_input)
Account 1001 has 2000, withdraw 1500 -> balance 500 (minimum). ... ok
test_case_3_invalid_input (test_case_3_invalid_input)
Account 1001 has 2000, transfer 100 to non-existing account 9999. ... ok
test_case_4_different_valid_scenario (test_case_4_different_valid_scenario)
1001 (2000), 1002 (3000); transfer 1000 from 1002 to 1001; enquire 1001. ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.254s

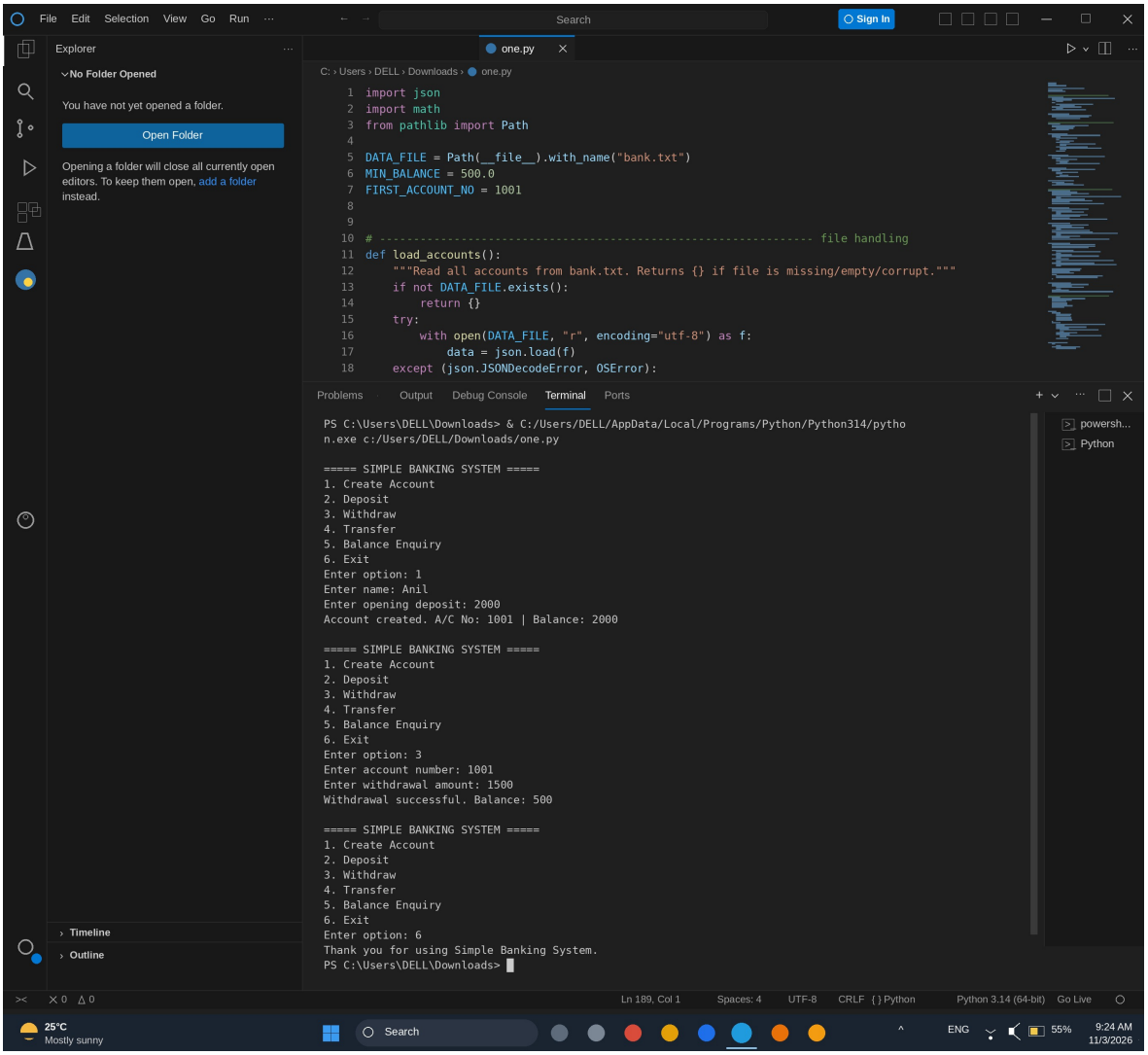
OK
```

8. Screenshots (Input and Output)

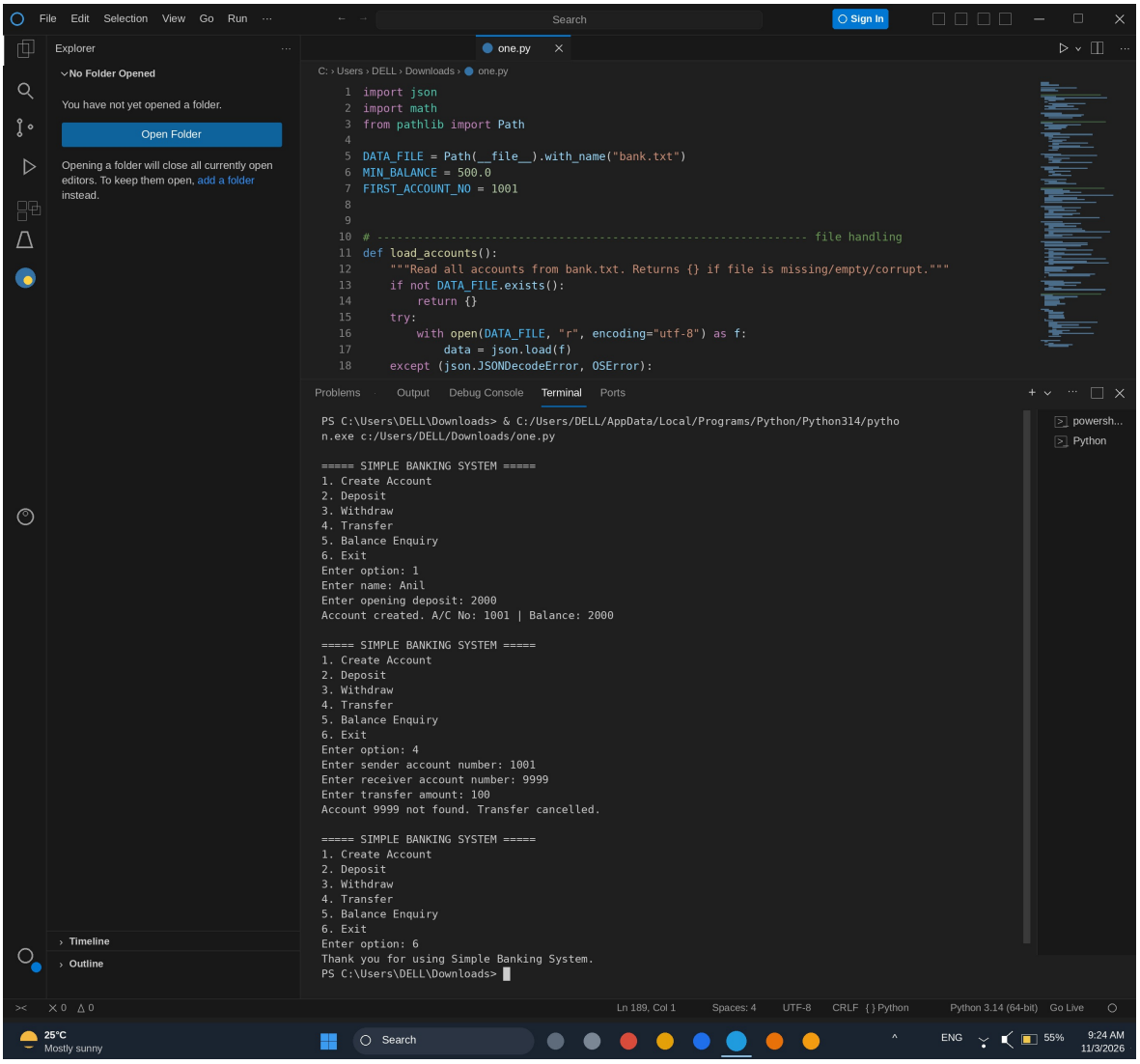
Test Case 1: Normal Input



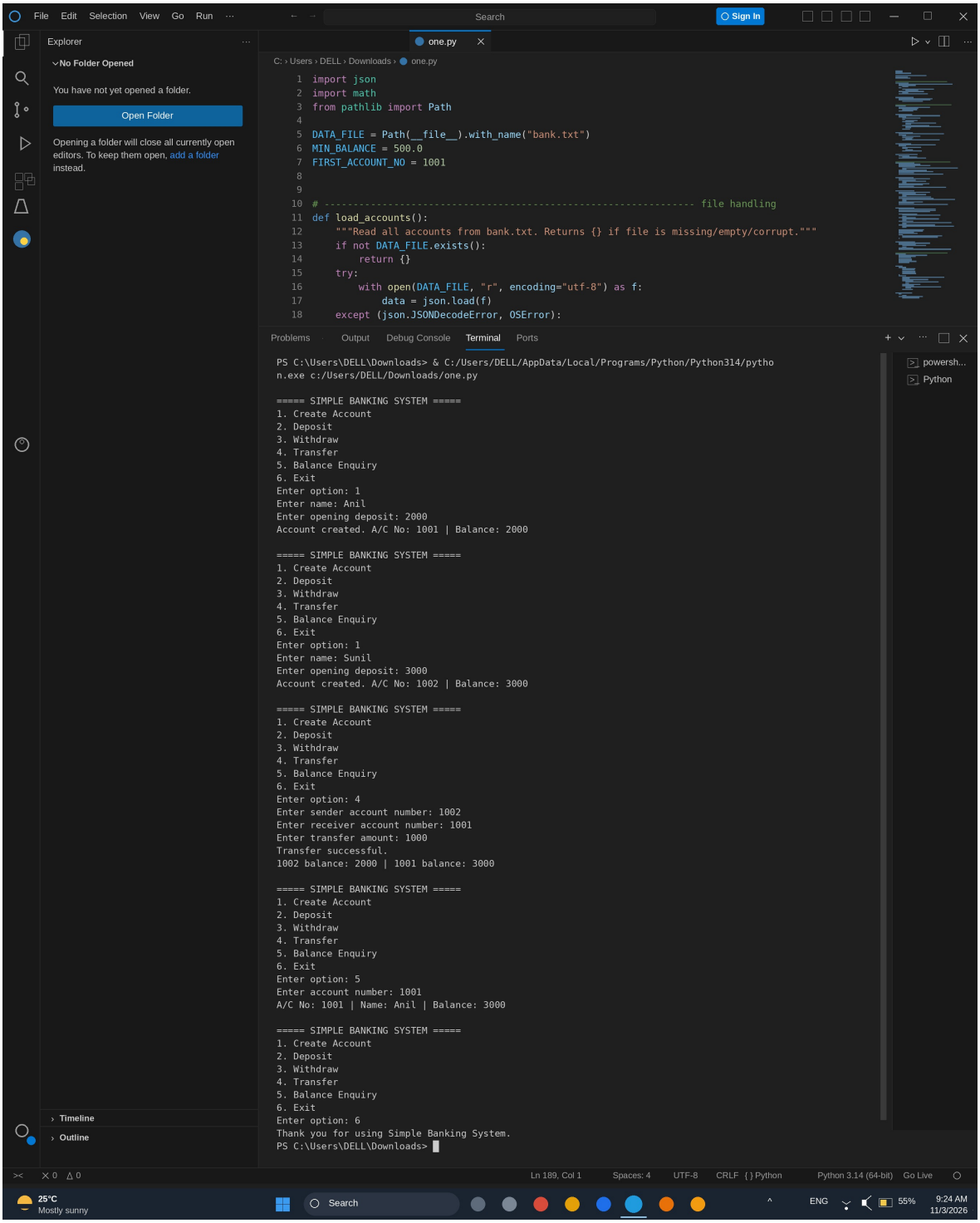
Test Case 2: Boundary / Minimum Input



Test Case 3: Invalid Input



Test Case 4: Different Valid Scenario



9. Conclusion

The Simple Banking System meets all the requirements of the problem statement. Accounts are created with unique numbers starting from 1001, the minimum balance of Rs.500 is always enforced, all inputs are validated, and all data is saved to and loaded from bank.txt. All four test cases pass. In future the project can be extended with passwords/PIN, transaction history, interest calculation and a graphical interface.

10. Source Code (one.py)

```
import json
import math
from pathlib import Path

DATA_FILE = Path(__file__).with_name("bank.txt")
MIN_BALANCE = 500.0
FIRST_ACCOUNT_NO = 1001


# ---------------------------------------------------------------- file handling
def load_accounts():
    """Read all accounts from bank.txt. Returns {} if file is missing/empty/corrupt."""
    if not DATA_FILE.exists():
        return {}
    try:
        with open(DATA_FILE, "r", encoding="utf-8") as f:
            data = json.load(f)
    except (json.JSONDecodeError, OSError):
        return {}
    return data if isinstance(data, dict) else {}


def save_accounts(accounts):
    """Write all accounts to bank.txt in JSON format."""
    with open(DATA_FILE, "w", encoding="utf-8") as f:
        json.dump(accounts, f, indent=2)


# ---------------------------------------------------------------- helpers / validation
def fmt(amount):
    """Show 2000.0 as 2000 and 2000.5 as 2000.50."""
    if float(amount).is_integer():
        return str(int(amount))
    return f"{amount:.2f}"


def amount_input(message):
    """Keep asking until the user enters a valid positive number."""
    while True:
        raw = input(message).strip()
        try:
            value = float(raw)
        except ValueError:
            print("Invalid amount. Please enter a number.")
            continue
        if not math.isfinite(value) or value <= 0:
            print("Invalid amount. Amount must be greater than 0.")
            continue
        return value


def account_input(message):
    """Keep asking until the user enters a whole-number account number."""
    while True:
        raw = input(message).strip()
        if raw.isdigit():
            return raw
        print("Invalid account number. Please enter digits only.")


def name_input(message):
    """Keep asking until the user enters a non-empty name."""
```

```
    while True:
        name = input(message).strip()
        if name:
            return name
        print("Name cannot be empty.")


def next_account_number(accounts):
    if not accounts:
        return FIRST_ACCOUNT_NO
    return max(int(no) for no in accounts) + 1


# ------------------------------------------------------------- banking operations
def create_account(accounts):
    name = name_input("Enter name: ")
    deposit = amount_input("Enter opening deposit: ")
    if deposit < MIN_BALANCE:
        print(f"Opening deposit must be at least Rs.{fmt(MIN_BALANCE)}. Account not created.")
        return
    acc_no = str(next_account_number(accounts))
    accounts[acc_no] = {"name": name, "balance": deposit}
    save_accounts(accounts)
    print(f"Account created. A/C No: {acc_no} | Balance: {fmt(deposit)}")


def deposit_money(accounts):
    acc_no = account_input("Enter account number: ")
    if acc_no not in accounts:
        print(f"Account {acc_no} not found.")
        return
    amount = amount_input("Enter deposit amount: ")
    accounts[acc_no]["balance"] += amount
    save_accounts(accounts)
    print(f"Deposit successful. Balance: {fmt(accounts[acc_no]['balance'])}")


def withdraw_money(accounts):
    acc_no = account_input("Enter account number: ")
    if acc_no not in accounts:
        print(f"Account {acc_no} not found.")
        return
    amount = amount_input("Enter withdrawal amount: ")
    if accounts[acc_no]["balance"] - amount < MIN_BALANCE:
        print(f"Withdrawal failed. Minimum balance of Rs.{fmt(MIN_BALANCE)} must remain.")
        return
    accounts[acc_no]["balance"] -= amount
    save_accounts(accounts)
    print(f"Withdrawal successful. Balance: {fmt(accounts[acc_no]['balance'])}")


def transfer_money(accounts):
    from_acc = account_input("Enter sender account number: ")
    to_acc = account_input("Enter receiver account number: ")
    amount = amount_input("Enter transfer amount: ")

    if from_acc not in accounts:
        print(f"Account {from_acc} not found. Transfer cancelled.")
        return
    if to_acc not in accounts:
        print(f"Account {to_acc} not found. Transfer cancelled.")
        return
    if from_acc == to_acc:
        print("Sender and receiver accounts must be different. Transfer cancelled.")
        return
    if accounts[from_acc]["balance"] - amount < MIN_BALANCE:
        print(f"Transfer failed. Minimum balance of Rs.{fmt(MIN_BALANCE)} must remain.")
        return

    accounts[from_acc]["balance"] -= amount
    accounts[to_acc]["balance"] += amount
    save_accounts(accounts)
    print("Transfer successful.")
    print(
        f"{from_acc} balance: {fmt(accounts[from_acc]['balance'])} | "
        f"{to_acc} balance: {fmt(accounts[to_acc]['balance'])}"
    )
```

```
def balance_enquiry(accounts):
    acc_no = account_input("Enter account number: ")
    if acc_no not in accounts:
        print(f"Account {acc_no} not found.")
        return
    acc = accounts[acc_no]
    print(f"A/C No: {acc_no} | Name: {acc['name']} | Balance: {fmt(acc['balance'])}")


# ------------------------------------------------------------------ main menu loop
def show_menu():
    print("\n===== SIMPLE BANKING SYSTEM =====")
    print("1. Create Account")
    print("2. Deposit")
    print("3. Withdraw")
    print("4. Transfer")
    print("5. Balance Enquiry")
    print("6. Exit")


def main():
    actions = {
        "1": create_account,
        "2": deposit_money,
        "3": withdraw_money,
        "4": transfer_money,
        "5": balance_enquiry,
    }
    while True:
        accounts = load_accounts()  # always work on the latest file data
        show_menu()
        choice = input("Enter option: ").strip()
        if choice == "6":
            print("Thank you for using Simple Banking System.")
            break
        action = actions.get(choice)
        if action is None:
            print("Invalid option. Please choose between 1 and 6.")
            continue
        action(accounts)


if __name__ == "__main__":
    try:
        main()
    except (KeyboardInterrupt, EOFError):
        print("\nProgram closed.")
```

Simple Banking System (PS No. 9)

Name: Tej Prakash Nishad | **ERP No.:** RU 26 01071

College: Rungta International Skills University – School of Information Technology

Programme: Bachelor of Computer Application (BCA)

Language: Python 3 | **Concepts:** Loops, Functions, Conditions, File Handling

1. Project Overview

A console-based banking application that lets customers open an account, deposit money, withdraw money, transfer money between accounts and check their balance. All account data is saved in a text file (`bank.txt`, JSON format) so it is still available after the program is closed.

2. Objectives

- Understand the problem statement and define it clearly.
- Build the solution using functions, loops, conditions and file handling.
- Enforce the banking rules (unique account numbers, minimum balance) and validate every input.
- Test the program with normal, boundary, invalid and alternate valid inputs.

3. Features

#	Feature	Description
1	Create Account	Takes name + opening deposit, gives a unique account number starting from 1001
2	Deposit	Adds money to an existing account
3	Withdraw	Removes money, but Rs.500 must always remain
4	Transfer	Moves money between two accounts (both must exist, minimum balance rule applies to sender)
5	Balance Enquiry	Shows account number, name and balance
6	Exit	Closes the program

4. Business Rules

- Account numbers start from 1001 and increase by 1 (1001, 1002, 1003 ...).
- Minimum balance of Rs.500 must always remain (also required as the opening deposit).
- Data is stored in `bank.txt` and loaded again on every menu cycle.

5. Input Validation

- Amount must be a number greater than 0 (letters, blank, negative, zero, `nan`/`inf` are rejected).
- Account number must contain digits only and must exist.
- Name cannot be empty.
- Sender and receiver of a transfer must be different accounts.
- A corrupt or empty `bank.txt` is handled safely (the program starts with no accounts).

6. Requirements

- Python 3.8 or higher (tested on Python 3.14)
- Only the standard library is used (`json`, `math`, `pathlib`) – nothing to install.

7. Project Structure

```
banking_system/
├── one.py            # complete project code
├── bank.txt          # data file (created/updated automatically)
├── test_banking.py   # automatic test file (all 4 test cases)
├── README.md         # this documentation
├── Banking_System_Report.pdf      # project report
├── Test_Case_Screenshots.pdf      # screenshots of every test case
└── screenshots/      # input/output screenshot of every test case
```

8. How to Run

1. Put `one.py` and `bank.txt` in the same folder.
2. Open the folder in VS Code (or any terminal).
3. Run: `python one.py`
4. Choose an option from the menu (1-6) and follow the prompts.

9. Functions Used

Function	Purpose
`load_accounts()` / `save_accounts()`	Read / write `bank.txt`
`amount_input()`, `account_input()`, `name_input()`	Input validation loops
`create_account()`	Open a new account
`deposit_money()` / `withdraw_money()`	Deposit / withdraw
`transfer_money()`	Transfer between accounts
`balance_enquiry()`	Show balance
`main()`	Menu loop

10. Test Cases (all 4 pass)

Test Case 1 – Normal Input

Input	Expected Output
Option 1, Name: `Anil`, Opening deposit: `2000`	`Account created. A/C No: 1001 | Balance: 2000`

Test Case 2 – Boundary / Minimum Input

Input	Expected Output
Account 1001 (balance 2000), Withdraw `1500`	`Withdrawal successful. Balance: 500`

Test Case 3 – Invalid Input

Input	Expected Output
Account 1001 (balance 2000), Transfer `100` to account 9999	`Account 9999 not found. Transfer cancelled.`

Test Case 4 – Different Valid Scenario

Input	Expected Output
Accounts 1001 (2000), 1002 (3000); Transfer 1000 from 1002 to 1001; Balance enquiry 1001	`Transfer successful. 1002 balance: 2000 | 1001 balance: 3000`

Screenshots of every test case are in the `screenshots/` folder and in `Test_Case_Screenshots.pdf`.

Run all test cases automatically

```
python test_banking.py
```

`test_banking.py` runs `one.py` for all 4 test cases (in a temporary folder, so your real `bank.txt` is not touched) and prints `OK` when all pass:

```
Ran 4 tests in 0.254s
OK
```

11. Sample `bank.txt`

```
{
  "1001": { "name": "Anil",  "balance": 3000.0 },
  "1002": { "name": "Sunil", "balance": 2000.0 }
}
```

Note: delete the contents of `bank.txt` (leave `{}`) before running Test Case 1 so the first account is 1001.

12. Author

Tej Prakash Nishad

ERP No.: RU 26 01071

Course: BCA – School of Information Technology

Rungta International Skills University

test_banking.py

Automatic test file for the Simple Banking System

```
"""
Test file for PS No. 9 - Simple Banking System
Author : Tej Prakash Nishad | ERP No. RU 26 01071
College: Rungta International Skills University

Runs one.py automatically for all 4 test cases and checks the expected output.
Every test uses its own temporary folder, so your real bank.txt is never changed.

Run:  python test_banking.py
"""
import shutil
import subprocess
import sys
import tempfile
import unittest
from pathlib import Path

PROJECT_FILE = Path(__file__).with_name("one.py")


def run_program(inputs):
    """Run one.py in a fresh temp folder, feed the inputs, return the console output."""
    with tempfile.TemporaryDirectory() as tmp:
        shutil.copy(PROJECT_FILE, Path(tmp) / "one.py")
        result = subprocess.run(
            [sys.executable, "one.py"],
            input="\n".join(inputs) + "\n",
            capture_output=True,
            text=True,
            cwd=tmp,
            timeout=30,
        )
        return result.stdout


class TestSimpleBankingSystem(unittest.TestCase):

    def test_case_1_normal_input(self):
        """Create account: Name Anil, opening deposit 2000."""
        out = run_program(["1", "Anil", "2000", "6"])
        self.assertIn("Account created. A/C No: 1001 | Balance: 2000", out)

    def test_case_2_boundary_minimum_input(self):
        """Account 1001 has 2000, withdraw 1500 -> balance 500 (minimum)."""
        out = run_program([
            "1", "Anil", "2000",        # setup: account 1001 with 2000
            "3", "1001", "1500",        # withdraw 1500
            "6",
        ])
        self.assertIn("Withdrawal successful. Balance: 500", out)

    def test_case_3_invalid_input(self):
        """Account 1001 has 2000, transfer 100 to non-existing account 9999."""
        out = run_program([
            "1", "Anil", "2000",        # setup: account 1001 with 2000
            "4", "1001", "9999", "100", # transfer to 9999
            "6",
        ])
        self.assertIn("Account 9999 not found. Transfer cancelled.", out)

    def test_case_4_different_valid_scenario(self):
        """1001 (2000), 1002 (3000); transfer 1000 from 1002 to 1001; enquire 1001."""
        out = run_program([
            "1", "Anil", "2000",        # setup: account 1001 with 2000
```

```
            "1", "Sunil", "3000",       # setup: account 1002 with 3000
            "4", "1002", "1001", "1000",  # transfer 1000 from 1002 to 1001
            "5", "1001",                # balance enquiry 1001
            "6",
        ])
        self.assertIn("Transfer successful.", out)
        self.assertIn("1002 balance: 2000 | 1001 balance: 3000", out)
        self.assertIn("A/C No: 1001 | Name: Anil | Balance: 3000", out)


if __name__ == "__main__":
    unittest.main(verbosity=2)
```